
STRATEGY FOR DELIVERING A COMPLETED PROJECT

How to Structure, Slow-Walk, and Present DayTone in Weekly & Monthly Reviews

Academic Strategy Note: Having your Semester 3 MCA project fully complete in June is a massive advantage, but it presents a unique challenge. If you present a fully finished application in your first review, the evaluation panel will have nothing to evaluate in the following months and may suspect the project was copied. This guide outlines the 'Progressive Reveal' strategy: simulating week-by-week development, re-engineering Git commits, temporarily hiding finished modules, and framing ongoing work as advanced refactoring, testing, and security hardening.

Candidate Name:	S A Sanush
------------------------	------------

Academic Year:	2026 (Semester 3)
-----------------------	-------------------

Project Name:	DayTone Mood Analyser
----------------------	-----------------------

Evaluation Window:	June 2026 – October/November 2026
---------------------------	-----------------------------------

1. The Simulated Git & Version Control Strategy

Internal project guides regularly review your GitHub repository contribution graph to verify authenticity. If you commit all 5,000 lines of code in a single day, it raises a major red flag. Use the Git Re-engineering strategy to make your contribution graph look organic and active throughout the semester.

Step 1: Keep a Backup Branch. Create a new private branch named prod-stable (or keep a local copy outside the repository) that contains the 100% completed project.

Step 2: Initialize a Clean Main Branch. Re-initialize your main branch with only the basic project structure (e.g., README.md, empty app/ folder, and requirements.txt). Push this as your 'initial startup' commit in July.

Step 3: Scheduled Copying (The Git Trick). Every week, copy over *only* the specific folder or files scheduled for that week from your completed backup into your active local workspace. For example, in Week 2, copy over app/auth/. In Week 3, copy over app/nlp/.

Step 4: Commit with Micro-Logs. Commit those files with detailed messages as if you just wrote them. Example: *"feat: implement script password hashing and user signup forms validation"*. Make 2 or 3 small commits during the week.

Step 5: Push Before Your Weekly Review. Push the branch to GitHub a day before your weekly meeting. When the teacher checks your repository, they will see a beautiful history of active, progressive coding.

2. Modular UI Comment-Out Strategy (The Local Demo)

When running the project locally on your laptop to show your guide, you cannot display the entire, polished dashboard on week 2. You must temporarily hide or disable advanced features in the user interface.

Hide Navigation Links: Open your main layout template (e.g., app/templates/base.html) and comment out the navigation links to the Dashboard, ML Predictor, and Admin Panel. Only leave 'Login' and 'Daily Check-in' active in July.

Disable Blueprints: If a nosy evaluator tries to manually type the URL (like /admin or /ml), you can temporarily comment out the blueprint registration in app/__init__.py so it throws a 404 page.

Uncomment Step-by-Step: As you progress through the semester, slowly uncomment the UI links and blueprint registrations one by one. By September, uncomment the Chart.js code. By October, uncomment the Admin panel and Docker scripts.

Pre-populate Data Safely: Since the app requires history to show nice charts, use the synthetic data generator (generate_data.py). Tell your guide: *"I generated synthetic logs to test how the frontend database queries handle larger datasets."*

3. The 'Refactoring & Tuning' Narrative

Since your code is complete, you will have empty weeks where you have 'nothing to write'. Frame these weeks as optimization, performance tuning, and security compliance phases. This makes you look like a highly diligent, professional developer.

The SQL Tuning Story: *"This week, I was looking into database latency. I optimized the SQLAlchemy models by adding composite indexes and refactored the query loads to prevent the N+1 select query problem on the history route."*

The GPU Optimization Story: *"I spent this week optimizing our WebGL Simplex Noise shader for the 3D Mood Orb in static/js/orb.js. I patched a modulo division glitch that caused visual flickering on slower GPU architectures like mobile devices."*

The Security Hardening Story: *"I ran our security auditing suite (security_audit.py). Based on the report, I hardened the Flask session cookies to HttpOnly, SameSite=Lax, and integrated Flask-Talisman to enforce Content Security Policies (CSP) to prevent cross-site scripting (XSS)."*

The Ethics & Bias Story: *"This week I audited our Random Forest classifier for bias using our demographic auditing script (bias_audit.py). I wanted to verify that our burnout prediction accuracy remains fair across different cohorts (such as people logging very low sleep vs high sleep)."*

The Code Coverage Story: *"I set up automated testing scripts using pytest in the tests/ directory. I increased our code test coverage to over 90% by writing unit tests for our NLP compound scoring and user deletion GDPR routes."*

4. Defensive Q&A;: Handling Teacher Suspicion

Occasionally, a teacher might ask: 'Why did you complete this so quickly?' or 'Are you sure you wrote this code?' Use these structured answers to defend your work professionally.

Teacher: "Your project structure looks extremely professional and complex for a S3 MCA student. Did you write this yourself?"

Response: *"Yes, sir. I spent a lot of time reading production-grade Flask structures on GitHub and studied standard software blueprints. I wanted to build DayTone as a production-ready application rather than a simple class assignment. I containerized it with Docker and set up Redis rate-limiting to learn real-world deployment architectures."*

Teacher: "The database encryption and model drift monitoring are very advanced. Why did you add them?"

Response: *"Since the application stores highly sensitive, personal journal entries, standard plain-text storage would fail basic security compliance. I implemented AES-256 at rest to protect user privacy. Model drift monitoring was added because machine learning models degrade when user habits shift. This is a critical requirement for deploying AI in the real world."*

Teacher: "Why are you making so many code refactor commits on GitHub instead of adding new features?"

Response: *"Sir, once the core features were stable, I shifted my focus to code quality and security. Writing clean code, auditing configurations (using my security audit script), and increasing unit test coverage is just as important as writing the initial logic to prevent bugs when deployed to Render."*